

A Major Project Mid-term Report on
SMART COLLEGE EVENT MANAGEMENT SYSTEM
Submitted in Partial Fulfillment of the Requirements for
the Degree of
BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING
Under Pokhara University

Submitted by:
Arpan Baniya, Roll No.211208
Abhinav Acharya, Roll No.211203
Md Tohid Reza, Roll No.211217

Under the supervision of:
Mr. Shree Krishna Yadav

Date: 22/01/2026

Department of Computer Engineering



NEPAL COLLEGE OF
INFORMATION TECHNOLOGY

Balkumari, Lalitpur, Nepal

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Nepal College of Information Technology (NCIT) for providing us the environment, guidance, and support required to undertake this major project. We would especially like to thank our Head of Department, Er. Amit K. Shrivastava, for his continuous encouragement, valuable supervision, and for offering us the opportunity to develop a fully functional software system aligned with real-world requirements.

We also extend our thanks to faculty members, friends, and everyone who contributed directly or indirectly to the successful preparation of this proposal.

ABSTRACT

The Smart College Event Management System addresses the critical challenge of inefficient event coordination and participation management in educational institutions. Traditional manual event planning processes suffer from poor communication, scheduling conflicts, low engagement, and lack of data-driven insights. This research presents an integrated web-based platform that leverages artificial intelligence to streamline event lifecycle management from creation to post-event analysis. The proposed solution implements a comprehensive full-stack architecture featuring a React-based frontend with Typescript, Express.js backend, and Mongo DB database. Real-time communication is facilitated through Socket.IO, enabling instant notifications and live event updates. The platform supports multi-role user management with hierarchical permissions for students, student administrators, and super administrators.

Expected impact includes significant improvement in event participation rates, reduction in administrative overhead, enhanced student engagement through personalized recommendations, and data-driven decision making for event planning. The system demonstrates how AI integration can transform traditional campus activities into smart, efficient, and engaging experiences while providing valuable insights for continuous improvement.

Keywords: *Event Management System, Artificial Intelligence, Web Application, Real-time Communication, Student Engagement*

TABLE OF CONTENTS

1. INTRODUCTION	1
1.1 PROBLEM STATEMENTS	1
1.2 PROJECT OBJECTIVES	2
1.3 SCOPE	2
2. LITERATURE REVIEW	4
3. METHODOLOGY	7
3.1 PROJECT METHODOLOGY	7
3.2 REQUIRED DIAGRAMS	12
3.2.1 USE CASE DIAGRAM.....	12
3.2.2 SYSTEM FLOWCHART.....	13
3.2.3 ACTIVITY DIAGRAM	14
3.3 ALGORITHMS USED	15
3.4 TOOLS USED.....	21
4. TEST CASES AND RESULTS	22
4.1 USER AUTHENTICATION AND EVENT CREATION	22
5. TASKS COMPLETED.....	24
6. TASKS REMAINING.....	25
7. PROJECT TASK AND TIME SCHEDULE.....	26
7.1 GANTT CHARTS.....	26
8. OUTPUT	30
9. REFERENCES	32

1. INTRODUCTION

The Smart College Event Management System with AI is a web-based platform designed to centralize and streamline all activities related to college events across various departments. It provides students with an organized and accessible way to explore social programs, technical events, sports competitions, workshops, seminars, and other campus activities. The system ensures that users always have up-to-date information about upcoming events, event descriptions, schedules, venues, and participation details.

For administrators, the system simplifies event coordination by offering tools to create, manage, update, and approve events efficiently. Student admins can also manage their respective events, track participants, and handle registrations or payments with ease. The integration of AI enhances the platform with features such as automated event categorization, smart recommendations, sentiment analysis on feedback, and an intelligent Chatbot to assist users with queries [1].

Overall, this project aims to support both students and staff by reducing manual work, minimizing communication gaps, and improving the overall event management workflow. It enhances convenience, accuracy, and efficiency while providing a secure and user-friendly environment for all college event operations.

1.1 PROBLEM STATEMENTS

Managing college events manually leads to several challenges:

- Students miss important events due to poor communication as event details are often scattered, incomplete, or delivered late.
- Manual registration and attendance tracking are time-consuming.
- There is no centralized platform for event approvals or student-admin roles.
- Real-time updates and participant counts are impossible with traditional methods.
- Payment handling for paid events becomes unorganized.
- Feedback is collected but not analyzed effectively and no intelligence exists to suggest events based on student interests.

Therefore, there is a need for a centralized, automated, AI-enhanced event management system that ensures smooth coordination among students, faculty, and administrators.

1.2 PROJECT OBJECTIVES

The objectives of smart college event management system are listed below:

- To develop a web-based system for managing all college events in one platform.
- To implement real-time features for instant updates, and integrate AI for chatbot queries, recommendations, and automation.
- To enable secure digital payments for paid events, simplify registration, and feedback processing.

1.3 SCOPE

The scope of a Smart College Event Management System encompasses various functionalities and aspects that contribute to the successful organization, management, and execution of events within an educational institution such as:

- Capability to manage various types of events, including academic conferences, cultural festivals, seminars, workshops, department programs, and sports activities.
- Creation and management of event specifics such as title, date, time, venue, images, categories, description, and assigned admins.
- Allow students, faculty, and other users to register for events and view registration status.
- Maintain detailed user profiles for participants, student admins, and super admins, ensuring personalized experiences and structured access.
- Provide automated reminders, email/notification updates, and event-related announcements for all registered participants.
- Enable communication channels such as integrated chatbot support, admin messages, and feedback submission.

- Include a robust Super Admin panel for monitoring all events, managing user roles, reviewing admin requests, generating insights, and viewing analytics.
- Assign different access levels (Normal User, Student Admin, Super Admin) with clear privileges to ensure proper control and secure management of events.
- AI-powered functionalities such as personalized event recommendations and sentiment analysis of feedback.
- Support secure login, verification, and activity logging to maintain a safe and reliable application environment.
- Provide an intuitive and responsive user interface with search, filtering, trending events, recommended events, and other usability features [4].

2. LITERATURE REVIEW

Event management systems have evolved significantly over the past decade as educational institutions shifted from manual record-keeping to digital automation. Earlier systems mainly focused on managing event schedules and registrations but lacked flexibility, scalability, and intelligent features [1].

Paper Based Event Management System

Many schools, colleges and organization use process of manual event management system. The process is time consuming as well as sometimes inefficient resulting in the managing the events. There is also a need for the files/records to be maintained by the event holders in hardcopy form and generate report at the end of event. It also needs space require to hold the files/ records of the events.

Existing College Event Portals

Several colleges used simple event portals that allowed only basic browsing and registration. These systems often stored data in spreadsheets or unstructured databases, which made event modifications, participant tracking, and report generation difficult. They were also unable to handle multiple event types or roles such as student admins [4].

Learning Management Systems (LMS) Influence:

Platforms like Moodle and Blackboard inspired structured data handling, user account management, and standardized interfaces. However, they were designed primarily for course management, not event workflows. Their complexity made them impractical for exclusive event management in colleges.

Eventmie

Eventmie Pro is an advanced, Laravel-based event management system that simplifies the process of organizing, hosting, and managing events. Its robust documentation serves as a guide for developers and users, ensuring effective implementation, customization, and optimization of the platform. This review outlines the key features, technical architecture, and use cases of Eventmie Pro, aligning them with the goals of this project proposal.

While Eventmie has established itself as a popular platform, its impact on the schools and colleges and its overall effectiveness in addressing the challenges of the educational institutes warrant further research. A comprehensive analysis of user experiences, transaction success rates, and the platform's influence on event planning and market trends would provide valuable insights into its role and potential for improvement.

Khelsetu

The Khel Setu project is a platform designed to streamline the organization and management of sports events. It enables event organizers to manage tasks like scheduling, participant registrations, payment processing, and venue details efficiently. Participants can register online, track updates, and access event information, while the admin dashboard offers comprehensive CRUD operations for event management. The goal of the project is to simplify the management process and make sports events enjoyable for all, whether for small tournaments or large-scale events.

The project has some small drawbacks that could be improved. The repository does not outline strategies for handling scalability, which is critical for managing large events.

AI in Event & Content Management:

Recent studies on AI-driven systems show how NLP models can classify content, recommend items, and analyze sentiment. RNNs, transformers, and pre-trained models like BERT have been used to understand user preferences and feedback patterns. Research also highlights the importance of predictive analytics—such as attendance forecasting—to help organizers plan resources efficiently [7].

Despite advancements, most college event systems still lack:

- Internal admin approval flows and AI categorization of events
- Personalized recommendations for students and Sentiment-based feedback analysis
- Role-based dashboards for student admins & super admins

This project fills these gaps by integrating event automation, role-based workflows, and AI components into a single unified system tailored specifically for educational institutions.

To implement AI components, the project will utilize a combination of publicly available datasets and institution-generated data:

1. Recommendation System Data

- Data Source: User–event interaction logs generated within the system.
- Collected Attributes: User interest categories, past registrations, event popularity.
- Technique: Hybrid filtering (content-based + collaborative filtering).

2. Chatbot Training Data

- Data Source: FAQ documents, event guidelines, previous event brochures.
- Model: TF-IDF/BERT embeddings for similarity-based query matching.

3. Sentiment Analysis

- Data Source: User feedback gathered from the system.
- Technique: Logistic Regression/BERT classifier trained to classify feedback as positive, neutral, or negative.

All datasets will be cleaned, labeled, normalized, and divided into training and testing sets. Model performance will be evaluated using accuracy, F1-score, and confusion matrices.

3. METHODOLOGY

3.1 PROJECT METHODOLOGY

The Smart College Event Management System was developed using the Incremental Model of software development, which combines elements of waterfall and iterative approaches. This methodology was selected due to the project's complexity, involving multiple subsystems including artificial intelligence integration, real-time communication, and multi-role user management. The incremental approach allowed for systematic development and testing of core functionality before adding advanced features, ensuring robustness at each development phase[3].

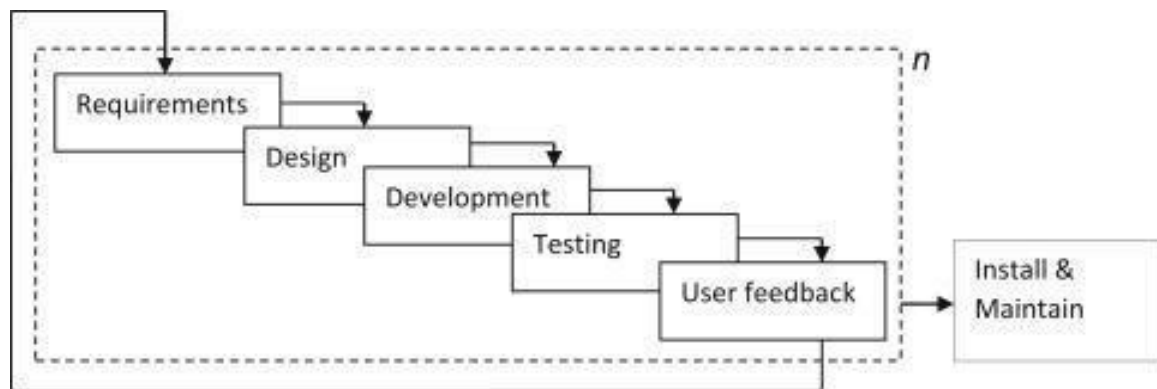


FIG 1: INCREMENTAL MODEL

Increment 1: Foundation Setup & Core Requirements

1 Planning and Requirements Gathered:

- Defined complete project objectives, scope, and constraints.
- Collected requirements for all user roles i.e. Normal User, Student Admin, and Super Admin.
- Identified system-wide features i.e. event management, authentication, notifications, analytics, payment system, and AI integration.
- Prioritized features to structure the entire development plan.

2 Design and Development:

- Designed full system architecture (frontend–backend communication, APIs,

database schema).

- Created database tables for users, events, admin requests, feedback, payments, and notifications.
- Set up backend environment, folder structure, authentication logic, and basic routing.

3 Testing and Deployment:

- Performed initial unit tests on backend architecture and database connectivity.
- Validated API responses and authentication routes.
- Deployed this foundational increment to a staging server for verification.

4 Subsequent Increments:

- Prioritized increments based on feedback.
- Continuously tested and deployed each increment to ensure seamless integration with existing functionalities.

At the end of increment 1, a complete architectural foundation was established. All requirements were collected, and the system structure is ready for actual feature development.

Increment 2: Core Event System & User Interface

1 Planning and Requirements Gathered:

- Gathered requirements for user-facing event features such as:
 - Event listings and Event details page
 - Search and filtering
- Planned user registration and login system behavior.

2 Design and Development:

- Developed the core frontend pages (Home, Events, Login, Signup, Profile layout).

- Implemented backend logic for:
 - Creating events, Listing events ,and Searching events
 - Implemented session management, validation, and protected routes.
 - Added image upload options and user profile.
- 3 Testing and Deployment:
- Conducted UI tests for responsiveness and functionality.
 - Tested API integration for event listing and user accounts.
 - Fixed design or data flow issues based on test feedback.

At the end of increment 2, a functional frontend–backend event system was completed. Users can register, log in, browse events, and search events.

Increment 3: Admin System, Notifications & Data Processing

- 1 Planning and Requirements Gathering:
- Gathered requirements for advanced functionalities:
 - Student Admin approval and Admin event management (CRUD)
 - Real-time updates using Socket.IO
 - Identify datasets required for AI modules and feedback analysis.
- 2 Design and Development:
- Implemented Super Admin features:
 - Approve / Reject events
 - Approve / Reject Student Admin requests
 - View logs, payments, and system analytics.
 - Developed Student Admin tools:
 - Send requests to admin
 - Edit event details

- Manage participants
 - Upload event resources
 - Added user registration, google calendar and maps
 - Added real-time features using Socket.IO:
 - Live participant count
 - Instant notifications & event updates
 - Real-time admin actions
 - Integrated email notifications using Sendgrid API.
 - Prepared datasets and connect feedback data to the database.
- 3 Testing and Deployment:
- Tested real-time features, integration stability, and admin privilege management.
 - Validated notifications, admin workflow, and event update syncing.
 - Deployed updated version to staging and gather feedback.

At the end of increment 3, the system supported complete event management, permission-based access, real-time updates, and notification handling.

Increment 4: Finalizing

- 1 Planning and Requirements Gathering:
- Finalize AI models needed:
 - Feedback Sentiment Analysis
- 2 Design and Development:
- Added history logs
 - Added AI modules:
 - Sentiment Analysis — predict positive/negative feedback

- Connected AI outputs to dashboards and user profiles.
- Added analytics dashboard for admin:
 - Revenue, requests, and Engagement statistics
- Improved UI/UX and optimize performance.
- Implemented chatbot for FAQs, event details, and suggestions.

3 Testing and Deployment:

- Tested AI predictions, chatbot accuracy, and system security.
- Conducted performance testing, load testing, and cross-device compatibility checks.
- Fixed bugs, finalized documentation, and prepared system for deployment.

At the end of increment 4, the system is feature-completed with real-time communication, AI functionalities, analytics, and polished UI/UX.

3.2 REQUIRED DIAGRAMS

3.2.1 USE CASE DIAGRAM

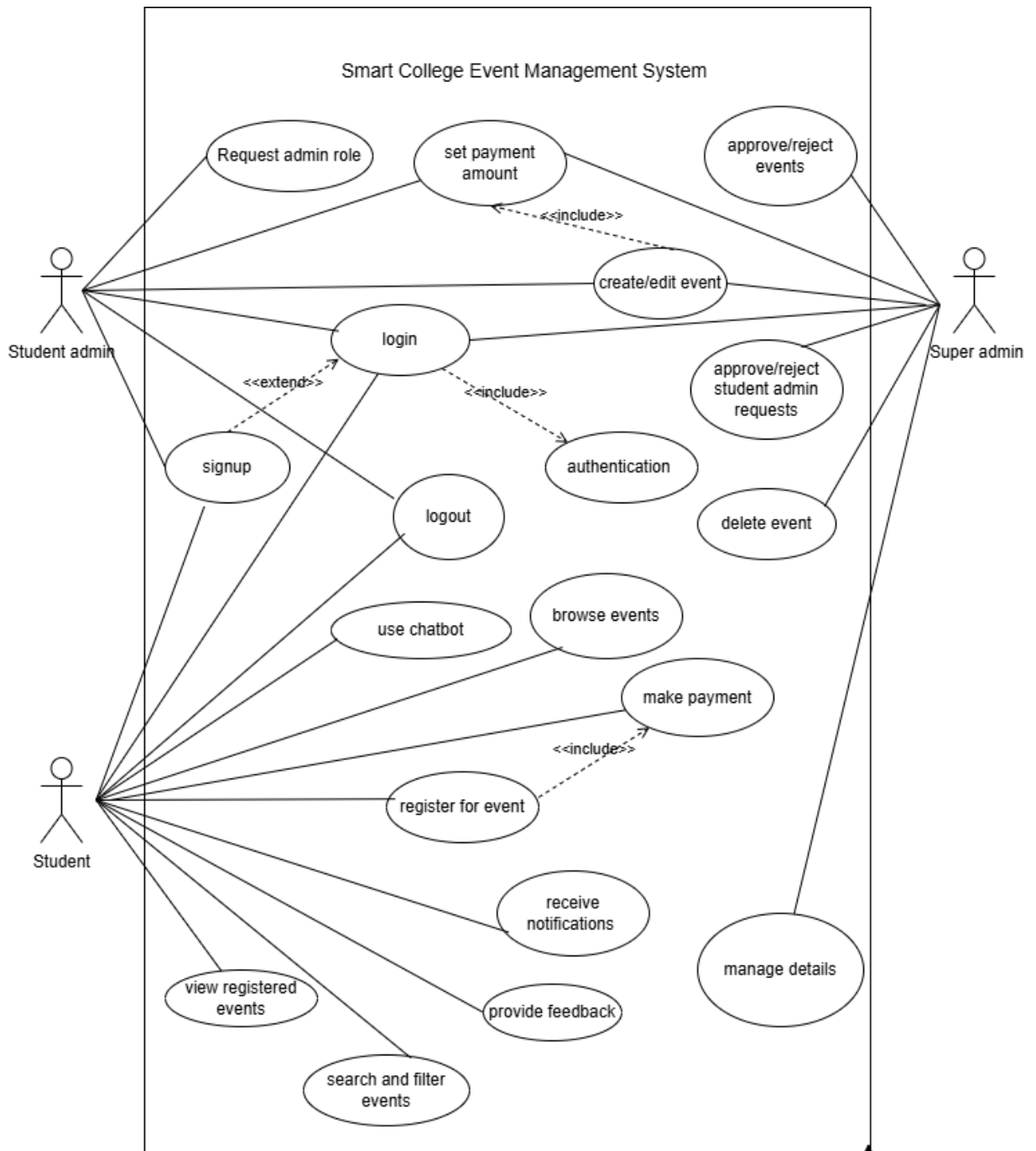


FIG 2: Use case diagram

3.2.2 SYSTEM FLOWCHART

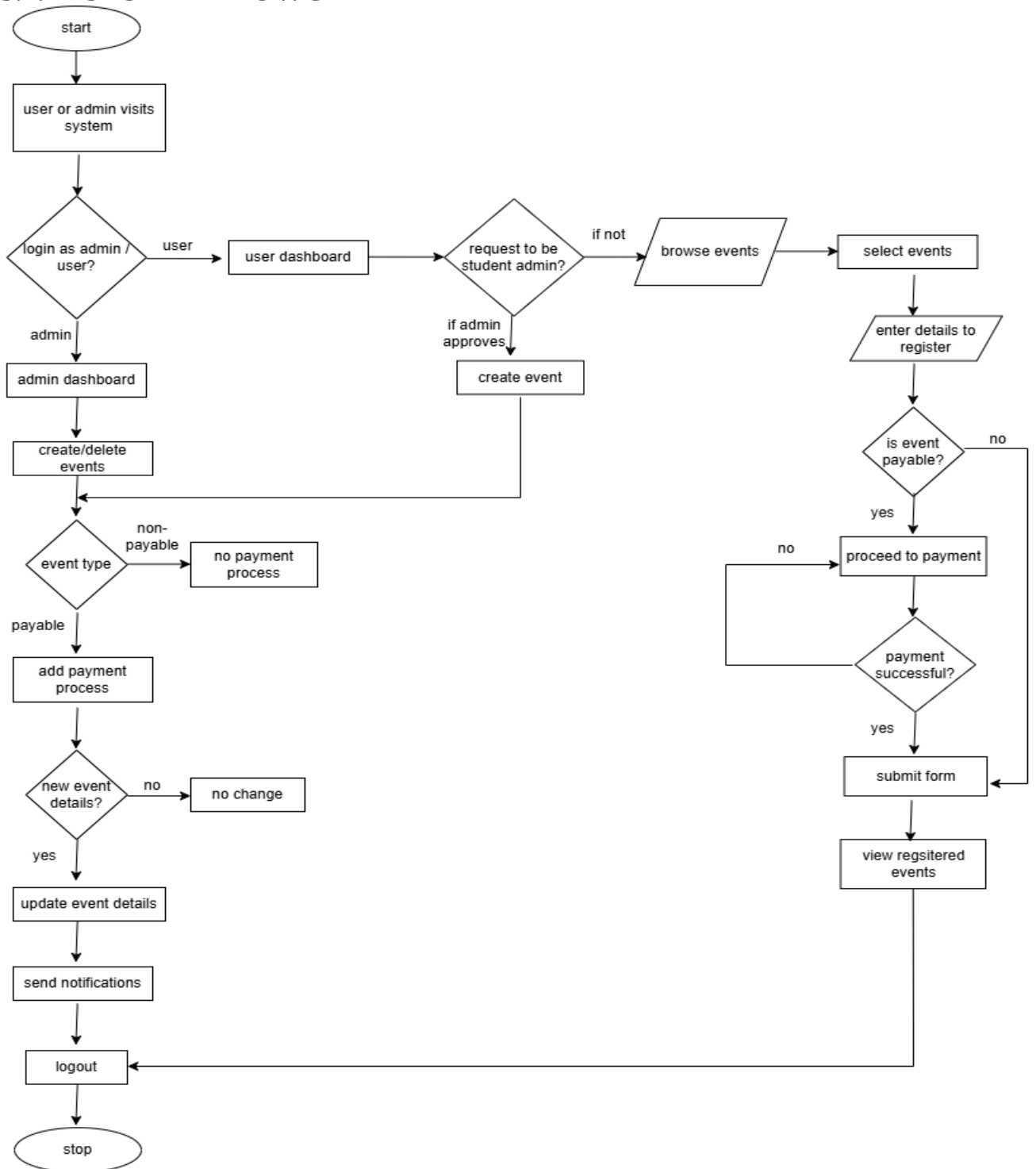


FIG 3: System flowchart

3.2.3 ACTIVITY DIAGRAM

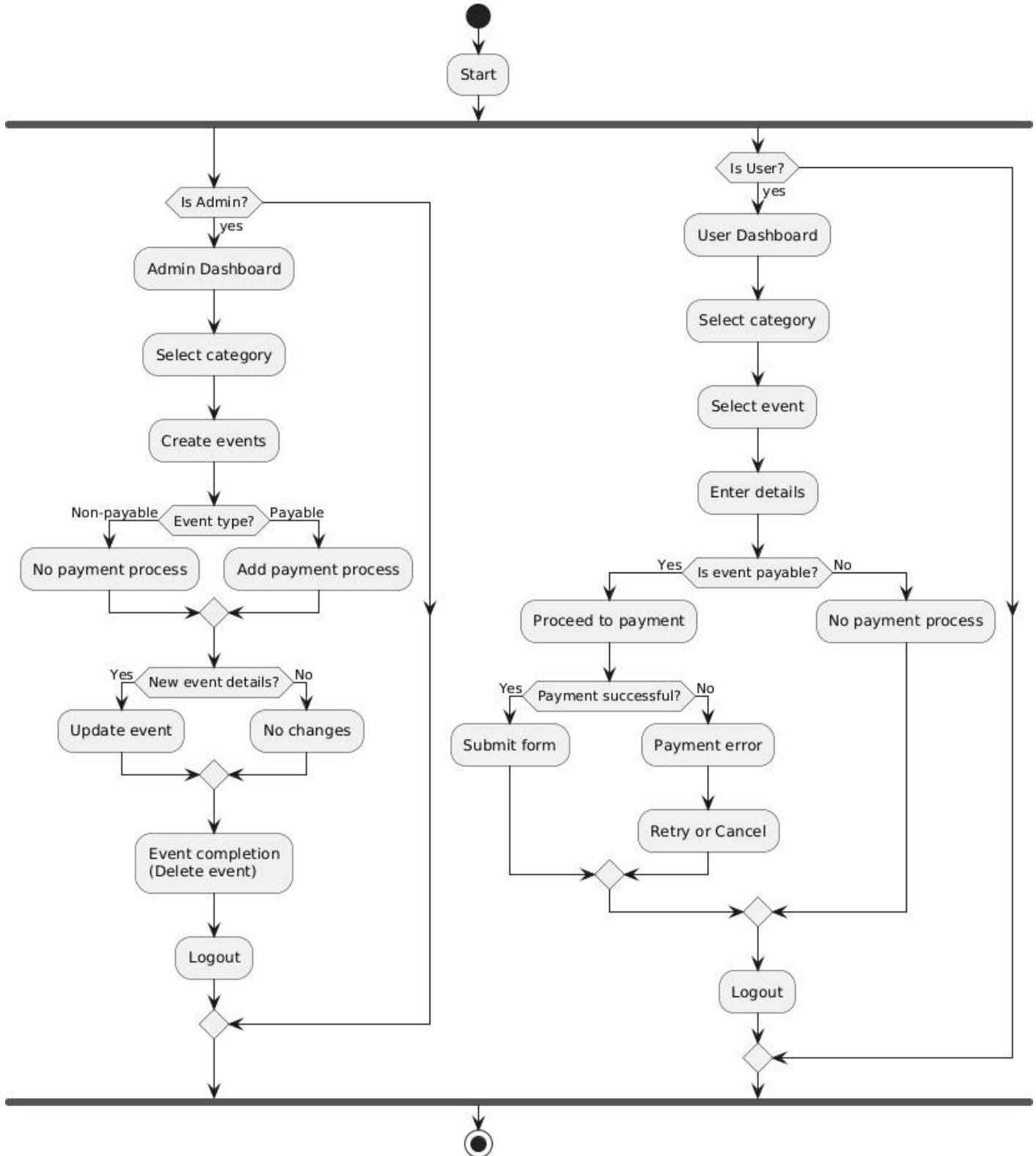


FIG 4: Activity Diagram

3.3 ALGORITHMS USED

1. Event Notification Algorithm

Algorithm: Push Notification Algorithm

- This algorithm delivers real-time updates to users regarding event reminders, registrations, cancellations, payment confirmations, and schedule changes.
- It ensures users receive timely and relevant notifications without unnecessary alerts.
- The system uses Socket.IO for instant in-app notifications and email API (SendGrid/Khalti) for scheduled reminders.

Uses

- Notify participants about changes in event schedules.
- Notify admins about new registration or student admin requests.

2. Event Search and Filtering

Algorithm: Search Algorithm (Binary Search)

Time Complexity: $T(n)=O(\log n)$

$mid = ((low + high)/2)$

If $A[mid] = Q \rightarrow$ return index

If $A[mid] < Q \rightarrow low = mid + 1$

If $A[mid] > Q \rightarrow high = mid - 1$

Where A = sorted list of events,

Q = target event to find,

Low, high= current search bounds

- Binary Search efficiently finds events when searching through a sorted list (by name, date, or category).
- The list is divided repeatedly to locate the target event, making it faster than linear search.

Uses

- Search events by title and filter events by date, category, and location.
- Improve search speed when dataset grows larger.

3. Event Recommendation Algorithm

Algorithm: Hybrid Recommendation System (Content-Based + Collaborative Filtering)

Content based Similarity (Cosine Similarity)

$$Sim(E_u, E_i) = \frac{E_u \cdot E_i}{\|E_u\| \|E_i\|}$$

Dot Product:

$$E_u \cdot E_i = \sum_{k=1}^n (E_{u,k} \times E_{i,k})$$

Magnitudes:

$$\|E_u\| = \sqrt{\sum_{k=1}^n E_{u,k}^2}, \quad \|E_i\| = \sqrt{\sum_{k=1}^n E_{i,k}^2}$$

Where:

E_u = user interest vector

E_i = event feature vector

Collaborative Filtering (User–User Similarity)

Pearson Correlation:
$$Sim(u, v) = \frac{\sum_i (r_{u,i} - \bar{r}_u)(r_{v,i} - \bar{r}_v)}{\sqrt{\sum_i (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_i (r_{v,i} - \bar{r}_v)^2}}$$

Where: $r_{u,i}$ =rating by user u for event i

$\bar{r}_u$ = avg rating of user u

Same for user v

Predicted Rating:
$$\hat{r}_{u,i} = \bar{r}_u + \frac{\sum_v Sim(u, v)(r_{v,i} - \bar{r}_v)}{\sum_v |Sim(u, v)|}$$

Hybrid Recommendation Score:

$$Score(u, e) = \alpha \cdot Sim_{content}(u, e) + (1 - \alpha) \cdot Sim_{collab}(u, e)$$

Where: α = weight (0–1)

$Sim_{content}$ = cosine similarity

Sim_{collab} = collaborative filtering score

- Suggests personalized events to users based on their interests and past interactions.
- Content-based filtering uses event categories and user-selected preferences.
- Collaborative filtering uses behavior patterns of similar users.

Uses

- Show “Recommended Events” on user dashboard.
- Suggest trending events based on participation patterns.

4. NLP-Based Chatbot Query Understanding Algorithm

Algorithm: TF-IDF / Embedding Similarity

Term Frequency (TF): $TF_{(t,d)} = \frac{(\text{count of } t \text{ in } d)}{(\text{total terms in } d)}$

Inverse Document Frequency (IDF): $IDF_t = \log\left(\frac{N}{df_t}\right)$

Final weight: $TFIDF_{t,d} = TF_{(t,d)} \cdot IDF_t$

Where: t= specific term,

N=Total number of documents in the dataset

df_t = Document frequency (number of documents that contain term t)

$TF_{(t,d)}$ =Normalized frequency of the word in the document

$TFIDF_{t,d}$ =Final TF-IDF weight for term t in document d

IDF_t =Inverse Document Frequency of term t

Naïve Bayes Classifier Formula:

$$P(\text{Class} | \text{Query}) \propto P(\text{Class}) \prod P(\text{word}_i | \text{Class})$$

Embedding-Based Chatbot (Similarity):

$$\text{sim}(A, B) = \frac{A \cdot B}{||A|| ||B||}$$

Where:

A=Embedding vector of the user query

B=Embedding vector of an intent example

A.B= Dot product of vectors A and B

$\text{sim}(A,B)$ = Cosine similarity score (range: -1 to 1)

- The chatbot understands user queries using text-processing algorithms.
- TF-IDF or BERT embeddings convert user queries into numerical vectors.
- The system compares these vectors with stored FAQ/question vectors to find the best answer.

Uses

- Answer student questions about events.
- Provide real-time support (timing, venue, and registration help).
- Suggest events based on user messages.[6]

5. Sentiment Analysis Algorithm

Algorithm: Logistic Regression / BERT Sentiment Classifier

Linear Score:

$$z = w^T * X + b$$

Probability (Sigmoid):

$$P(Positive) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Where: X = input vector of feedback,

w = weights,

b = bias,

σ = sigmoid function

Decision Rule:

Positive: $P > 0.6$

Neutral: $0.4 \leq P \leq 0.6$

Negative: $P < 0.4$

- Processes user feedback and classifies it as: Positive, Neutral, or Negative.
- Helps admins evaluate event success and student satisfaction.

Uses

- Display event sentiment score.
- Generate analytics for the dashboard.
- Identify events that students prefer.[7]

6. Real-Time Update Synchronization Algorithm

Algorithm: Socket Event Broadcasting Algorithm

- Ensures all users see live updates without refreshing the page.
- Uses event broadcasting to update:
 - Participant count and Event detail changes
 - Admin actions

Uses

- Real-time participant count display.
- Instant updates when admin edits event details.

3.4 TOOLS USED

TOOLS	PURPOSE
Visual Studio Code	IDE/ Code editor
React.js, HTML5, Tailwind CSS,typescript	Frontend
Node.js, Express.js,typescript	Backend
JWT (JSON Web Token)	Authentication and secure access control
Postman	API testing and verification
Git & GitHub	Version control and project repository
bcrypt	Security(password hashing)
MongoDB	NoSQL database
Socket.io	Real time communication
Sendgrid	Email service
Python	NLP Chatbot, Sentiment Analysis, Recommendation System
MS Word	Documentation

TABLE 1: TOOLS USED

4. TEST CASES AND RESULTS

4.1 USER AUTHENTICATION AND EVENT CREATION

Test Case ID	Test Description	Input Data	Expected Result	Actual Result	Test Status
AUTH-001	User registration with valid data	Email: user@test.com , Password: "Test@123", Name: "John Doe"	Successful registration, user created	User successfully created	PASSED
AUTH-002	Password length validation (less than 7 chars)	Email: user@test.com , Password: "Test@1"	Error message: "Password must be at least 7 characters"	Error message displayed	PASSED
AUTH-003	Password missing special character	Email: user@test.com , Password: "Test1234"	Error message: "Password must contain at least one special character"	Error message displayed	PASSED
AUTH-004	Invalid email format validation	Email: "invalid-email", Password: "Test@123"	Error message: "Invalid email format"	Error message displayed	PASSED
AUTH-006	Empty field submission validation	Email: "", Password: ""	Error messages for required fields	Error messages displayed	PASSED
AUTH-007	User login with correct credentials	Email: user@test.com , Password: "Test@123"	Successful login, JWT token generated	Login successful, token received	PASSED

Test Case ID	Test Description	Input Data	Expected Result	Actual Result	Test Status
AUTH-008	User login with incorrect credentials	Email: user@test.com , Password: "wrongpass"	Error message: "Invalid credentials"	Error message displayed	PASSED
EVENT-001	Mandatory field validation - missing event name	Name: "", Date: "2024-12-25", Time: "10:00", Venue: "Auditorium", Description: "Test event"	Error: "Event name is required"	Error message displayed	PASSED
EVENT-002	Mandatory field validation - missing date	Name: "Test Event", Date: "", Time: "10:00", Venue: "Auditorium", Description: "Test event"	Error: "Date is required"	Error message displayed	PASSED
EVENT-003	Invalid date entry (past date)	Name: "Test Event", Date: "2020-01-01", Time: "10:00", Venue: "Auditorium", Description: "Test event"	Error: "Date cannot be in the past"	Error message displayed	PASSED
EVENT-004	Empty field submission - all fields empty	All fields empty	Multiple validation errors	Validation errors displayed	PASSED
EVENT-005	Duplicate event handling	Same event details as existing event	Error: "Event already exists"	Error message displayed	PASSED
EVENT-006	Successful event creation with valid inputs	Name: "Valid Event", Date: "2024-12-25", Time: "10:00", Venue: "Auditorium", Description: "Valid description"	Event created successfully	Event created in database	PASSED

5. TASKS COMPLETED

- **Multi-Role Authentication:** Implemented complete authentication system supporting user, student_admin, and super_admin roles with hierarchical permissions.
- **JWT Token Management:** Developed secure JWT-based session management with token expiration, refresh mechanisms, and secure storage.
- **Password Security:** Integrated bcrypt for secure password hashing with salt generation and resistance to brute force attacks.
- **Session Management:** Implemented express-session with MongoDB store for persistent session management across server restarts
- **Role-Based Access Control (RBAC):** Created comprehensive permission system with middleware for route protection based on user roles.
- **Complete CRUD Operations:** Implemented full Create, Read, Update, Delete functionality for events with proper validation and error handling.
- **Image Upload System:** Implemented secure file upload functionality with Multer middleware for event images and resources.
- **Advanced Search Functionality:** Developed text-based search with MongoDB text indexing for efficient event discovery.
- **Multi-Criteria Filtering:** Implemented filtering system for date, category, location, and event type with real-time updates.
- **Search Result Ranking:** Created relevance-based search result ranking with configurable weight factors for different fields.
- **Component Library Creation:** Developed reusable UI components using Radix UI primitives with Tailwind CSS for consistent design.
- **Responsive Design Implementation:** Created mobile-first responsive design ensuring optimal user experience across all device sizes.
- **State Management:** Implemented React Query for server state management

with caching, background updates, and optimistic updates.

- Event Broadcasting: Created real-time event broadcasting for participant updates, status changes, and admin actions.
- Interactive Dashboard: Created comprehensive user dashboard with personalized event recommendations and activity tracking.
- Admin Interface: Developed specialized admin dashboard with analytics, user management, and event moderation capabilities.
- User Profile Building: Created dynamic user interest profiles based on past interactions, preferences.
- AI Chatbot Implementation (Natural Language Processing): Integrated TF-IDF vectorization for query understanding and FAQ matching.
- Multi-Channel Notifications: Implemented in-app notifications, email notifications via SendGrid, and real-time updates.

6. TASKS REMAINING

- Integration of khalti payment.
- Chatbot integration.
- Tiesheet generation for team and individual events.
- Sendgrid notification for each instance.
- Certificate generation for winners of specific event.
- Recommendation system for users.

7. PROJECT TASK AND TIME SCHEDULE

7.1 GANTT CHARTS

Date	26 th December	27 th December	28 th December	29 th December	30 th December	31 st December	1 st January
Requirement Analysis							
Planning							
Documentation							
Coding							
Testing							

FIG 5: Gantt chart for Increment 1

Index:

Completed Task	
Pending Task	

Date	2 nd January	3 rd January	4 th January	5 th January	6 th January	7 th January	8 th January
Requirement Analysis							
Planning							
Documentation							
Coding							
Testing							

FIG 6: Gantt chart for Increment 2

Index:

Completed Task	
Pending Task	

Date	9 th January	10 th January	11 th January	12 th January	13 th January	14 th January	15 th January
Requirement Analysis							
Planning							
Documentation							
Coding							
Testing							

FIG 7: Gantt chart for Increment 3

Index:

Completed Task	
Pending Task	

Date	16 th January	17 th January	18 th January	19 th January	20 th January	21 st January	22 nd January
Requirement Analysis							
Planning							
Documentation							
Coding							
Testing							

FIG 8: Gantt chart for Increment 4

Index:

Completed Task	
Pending Task	

8. OUTPUT

The outputs of the Smart College Event Management System are designed to enhance accessibility, streamline event management, and improve overall user experience through AI-driven automation and real-time communication.

1. Fully Functional Web Application

An online platform where students, event organizers, and administrators can:

- Browse, filter, and search events using categories, dates, or keywords.
- Register free with real-time confirmation.
- View live updates such as schedule changes.

2. Comprehensive Event Management Dashboard

A centralized admin interface equipped with:

- Tools to create, update, approve, reject, or delete events.
- Management of student admin requests, participant lists, and payments.
- Tabs for viewing feedback, sentiment analysis, and system logs.
- This dashboard provides transparency, reduces workload, and improves coordination.

3. Secure and Scalable System Architecture

The system delivers:

- Secure authentication using JWT, password hashing, and role-based access.
- A reliable backend capable of handling multiple events, large attendance, and high traffic.
- Database structures optimized for fast querying and scalable growth. This guarantees long-term sustainability and institutional-level reliability.

4. Real-Time Communication and Notifications

Through Socket.IO, Email APIs, and Google Calendar, the platform provides:

- Instant notifications for registrations, cancellations, approvals, and schedule changes.
- Real-time participant count and live event modifications.

5. AI-Powered Enhancements

The system includes multiple AI-driven outputs such as:

- NLP Chatbot for FAQs, and event guidance.
- Sentiment analysis on feedback to evaluate event quality.

6. Feedback & Reporting System

A structured module enabling:

- Users to submit reviews and ratings.
- AI to interpret feedback sentiment (positive/neutral/negative).
- Admins to view clear reports, insights, and trends.
- Identification of highly engaging or repeatedly successful events.[6]

By delivering these outputs, the system becomes an essential digital platform for academic institutions. It simplifies operations, increases engagement, reduces manual work, and provides advanced AI-driven insights—ultimately creating a more connected, informed, and efficient college environment.

9. REFERENCES

- [1] N. Alenzi, “Event Management System,” *Prezi*, Dec. 9, 2019. [Online]. Available: <https://prezi.com/p/99q8qz2xp88m/event-management-system/> [Accessed: Dec. 2023].
- [2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 8th ed. New York, NY, USA: McGraw-Hill, 2014.
- [3] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [4] N. K. Sharma and R. Mahapatra, “Design and Development of a Web-Based Event Management System,” *International Journal of Computer Applications*, vol. 178, no. 21, pp. 15–20, Apr. 2019.
- [5] K. K. Aggarwal and Y. Singh, *Software Engineering*, 3rd ed. New Delhi, India: New Age International, 2008.
- [6] S. R. Kumar and S. C. Sahu, “Event Recommendation System Using Machine Learning,” *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 3, pp. 250–258, 2021.
- [7] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *arXiv preprint arXiv:1810.04805*, 2018.